

Node.js Production Debugging

Full interview-ready notes — CPU profiling, memory leaks, event loop, tooling

PART A — CPU Profiling & Flame Graphs (recap)

Before touching anything: never restart first — establish a baseline (which instances? CPU vs memory? event loop delay?). One instance degraded = usually a leak. All instances elevated together = a code path being hit more often, not a leak.

Situation	Tool	Why
Reproduce locally	clinic flame -- node server.js	Fresh process under generated load
Live issue, happening now	0x --pid <pid>	Attaches to existing process, no restart
Already happened, gone	Continuous profiling (Pyroscope)	Always-on recorder, query past window

Flame graph: built from CPU samples taken every few ms. Y axis = call stack depth (root at bottom, leaf at top). X axis = frequency across samples, NOT duration of one call.

Mnemonic: "Flame graph = voting, not a stopwatch."

Pattern	Meaning	Fix location
Wide plateau at TOP	That leaf function itself is the CPU cost (e.g. JSON.stringify on a big payload)	Optimize that function directly
Wide block in MIDDLE	Parent isn't costly itself — its children (above it) add up to its width	Look at what it calls, not the parent
GC frames (v8::internal::Heap::*) wide/frequent	Not app logic at all — excessive allocation is causing heap pressure	Reduce allocations, reuse objects

Classic bug: logging middleware that does `logger.info({ body: req.body, headers: req.headers })` on every request — the logger calls `JSON.stringify` on the full objects internally, so at high traffic this becomes a wide `JSON.stringify` plateau. Fix: log only `contentLength`, not the full objects.

PART B — Memory Leaks & Heap Snapshots

What is a leak: an object whose job is done, but something still references it, so the garbage collector can't remove it. Analogy: a file sent to Trash, but a desktop shortcut still points to it — the file can't be fully deleted while that shortcut exists.

How to catch it — heap snapshot comparison: take two snapshots (`writeHeapSnapshot()`) a few minutes apart under normal load. Load both into Chrome DevTools → Memory → Comparison view, sort by Delta (how much each object type's count grew). Small growth in Array/String is normal. A large, one-directional growth in a single type (e.g. EventEmitter count +328) is the leak signal.

Mnemonic: "Delta = how much new stuff appeared that wasn't there before — the bigger the delta, the more suspicious."

LCC — the three common leak patterns:

Letter	Pattern	Signature in heap snapshot	Fix
L — Listener	'on()' added repeatedly, never removed	EventEmitter count keeps growing	Use <code>once()</code> , or <code>removeListener()</code> explicitly
C — Closure	A returned function retains a big object it doesn't fully need	Closure instances referencing large objects	Extract only the fields the inner function needs
C — Cache	Map/object grows, entries never evicted	Map/object size grows monotonically, never shrinks	Add an eviction policy (max size, LRU, TTL)

Interview 3-liner: "The three most common Node.js memory leak patterns are listener accumulation, closure leaks holding onto large objects unnecessarily, and unbounded caches with no eviction. All three show up in a heap snapshot comparison as an object type whose count keeps growing across snapshots without ever shrinking."

PART C — Event Loop, Live Debugging & Other Tools

Event loop delay: a different symptom from CPU — latency is high while CPU looks normal. Analogy: a single-lane road (Node is single-threaded) — one car stuck (a synchronous heavy task) jams everyone behind it, even if the engine (CPU) itself isn't overworked.

`monitorEventLoopDelay({resolution:10})` measures lag (p99 > 50ms = a problem). `async_hooks` pinpoints which specific async operation is slow.

Mnemonic: "CPU is cool, latency is hot → the road is jammed, not the engine."

Chrome DevTools remote debugging: `kill -USR1 <pid>` enables the V8 inspector on a live process without restarting. Tunnel it privately: `ssh -L 9229:localhost:9229 user@host`, then open `chrome://inspect`. Never expose the inspector port publicly — it has no authentication, so anyone reaching it can execute code in the process.

Core dumps + llnode: post-mortem analysis for crashes/OOM. Enable with `ulimit -c unlimited` beforehand. After a crash, `llnode node -c core.<pid>` lets you run `v8 bt` (backtrace) and `v8 heapstats` to see the state at the moment of failure.

Continuous profiling (Pyroscope): runs always-on (1-3% overhead), so you can query any past time window later — like CCTV footage. Lets you diff a flame graph from before vs. after a deploy to catch gradual regressions.

Production considerations:

- Set `--max-old-space-size` explicitly — Node defaults to ~1.5GB heap regardless of container size; without this a leak consumes all RAM before the OOM killer even fires.
- Store heap snapshots off-instance (e.g. S3) — writing a large file to a disk that's already under memory pressure can make things worse.
- Overlay deploy markers on the profiling dashboard — a leak that started right after a deploy is almost always caused by that deploy.
- Profile under real production load — a profile from a quiet, low-traffic instance tells you almost nothing; V8's JIT behaves differently under sustained load.

Full Interview Question Bank

Q1. What's the difference between CPU profiling and memory profiling — when would you use each?

CPU profiling (flame graphs) finds which functions consume CPU cycles. Memory profiling (heap snapshots) finds which objects accumulate and never get freed. Use CPU profiling for high-CPU/high-latency symptoms, memory profiling when RSS/heap grows steadily over time.

Q2. Explain what a flame graph shows. What do the X and Y axes represent?

X axis = frequency across profiler samples (not duration). Y axis = call stack depth, root at bottom, currently-executing leaf at top.

Q3. How would you detect a memory leak in a live Node.js process without restarting it?

Expose a protected debug endpoint that calls `writeHeapSnapshot()`, take two snapshots minutes apart under normal load, then compare them in Chrome DevTools sorted by Delta.

Q4. What's the difference between high CPU and high event loop delay? Can you have one without the other?

Yes. High CPU with low latency can happen with parallel async work spread across cores/IO. Low CPU with high latency happens when one synchronous operation blocks the single-threaded event loop, even though total CPU usage looks low.

Q5. What are common causes of memory leaks in Node.js?

Listener accumulation (event listeners added but never removed), closures holding references to large objects they don't need, and unbounded caches with no eviction policy (LCC).

Q6. Why shouldn't you expose the V8 inspector port publicly?

It has no built-in authentication — anyone who can reach it can execute arbitrary code in the process. Access it only via SSH tunnel or an internal VPN-gated network.

Q7. What's `--max-old-space-size` and why set it explicitly in a container?

It sets V8's heap size limit. Node defaults to ~1.5GB regardless of the container's actual memory limit, so without setting it explicitly a leaking process can consume all available RAM before the OS OOM killer even reacts, causing cascading failures.

Q8. For a CPU spike happening right now in production, would you use `clinic flame -- node server.js`? Why or why not?

No — that command starts a brand-new process, losing the exact state causing the spike. Instead attach to the existing process with `0x --pid` (or send `SIGUSR1` to enable the inspector), which profiles the already-running process without restarting it.

Q9. A flame graph shows a wide block in the middle of the stack but the top splits into several smaller blocks. What does this mean?

The middle function isn't costly by itself — it's calling multiple expensive children whose combined time adds up to its own width. The fix targets what it calls (e.g. a costly downstream function), not the middleware itself.

Q10. You see frequent, wide `v8::internal::Heap::*` frames in a CPU profile. What does that indicate?

The CPU spike isn't from application logic — it's from garbage collection working overtime because the app is allocating too many objects too fast (heap pressure). The fix is reducing allocations, not optimizing a specific function.

Q11. How do you track event loop delay, and what indicates a problem?

Use `perf_hooks`' `monitorEventLoopDelay()`, reporting p50/p99/max periodically. A p99 above ~50ms indicates synchronous work (large `JSON.parse`, sync file reads, unbatched crypto, regex backtracking) is blocking the loop.

Q12. What tool would you use to analyze a Node.js process after it crashed with OOM?

Enable core dumps ahead of time (`ulimit -c unlimited`), then analyze the resulting core file post-mortem with `llnode` — using `v8 bt` for the JavaScript backtrace and `v8 heapstats` for heap state at the moment of the crash.

Q13. What's the benefit of continuous profiling over ad-hoc profiling?

Ad-hoc profiling only catches problems you already suspect and are actively investigating. Continuous profiling (e.g. Pyroscope) runs always-on at low overhead, letting you retroactively query any past time window and diff flame graphs from before/after a deploy to catch regressions you didn't know to look for.